AI-ASSISTED CODING

ASSIGNMENT 6.3

NAME:V.SIDDARTHA
HALL:2303A52062
BATCH:45

Task Description #1: Classes (Student Class)
Scenario
You are developing a simple student information management module.
Task
- **Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.**
- **The class should include attributes such as name, roll number, and branch.**
- **Add a method display_details() to print student information.**
- **Execute the code and verify the output.**
- **Analyze the code generated by the AI tool for correctness and clarity.**

Expected Output #1
- **A Python class with a constructor (__init__) and a display_details() method.**
- **Sample object creation and output displayed on the console.**
- **Brief analysis of AI-generated code.**

PROMPT
Create a Python class named Student with attributes name, roll_number, and branch. Add a method display_details() to print student information. Also create a few sample objects and display their details.

CODE:
```
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Name: {self.name}, Roll Number: {self.roll_number}, Branch: {self.branch}")
# Sample objects
student1 = Student("Alice", "101", "Computer Science")
student2 = Student("Bob", "102", "Mechanical Engineering")
student3 = Student("Charlie", "103", "Electrical Engineering")
# Display details of the students
student1.display_details()
student2.display_details()
```

student3.display_details()

OUTPUT:

```
student3.display_details()
#This code defines a Student class with attributes and a method to display details, and
```

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

Name: Charlie, Roll Number: 103, Branch: Electrical Engineering
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

JUSTIFICATION:

This code defines a Student class with attributes and a method to display details, and then creates sample student objects to demonstrate its functionality.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- **Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.**
- **Analyze the generated loop logic.**
- **Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).**

Expected Output #2

- **Correct loop-based Python implementation.**
- **Output showing the first 10 multiples of a number.**

PROMPT:

Write a Python function that prints the first 10 multiples of a given number using a for loop. Include example usage.

CODE:

```
def print_multiples(number):
    print(f"First 10 multiples of {number}:")
    for i in range(1, 11):
```

```
    print(number * i)
# Example usage
print_multiples(5)
```

OUTPUT:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

Name: Charlie, Roll Number: 103, Branch: Electrical Engineering
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/6.3 assign.py/TASK2.PY"
First 10 multiples of 5:
5
10
15
20
25
30
35
40
45
50
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

JUSTIFICATION:

This code defines a function print_multiples that takes a number as input and prints the first 10 multiples of that number using a for loop. The example usage demonstrates how to call the function with the number 5.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- **Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups**
(e.g., child, teenager, adult, senior).
- **Analyze the generated conditions and logic.**
- **Ask the AI to generate the same classification using alternative conditional structures (e.g.,**
simplified conditions or dictionary-based logic).

Expected Output #3

- **A Python function that classifies age into appropriate groups.**
- **Clear and correct conditional logic.**
- **Explanation of how the conditions work.**

PROMPT

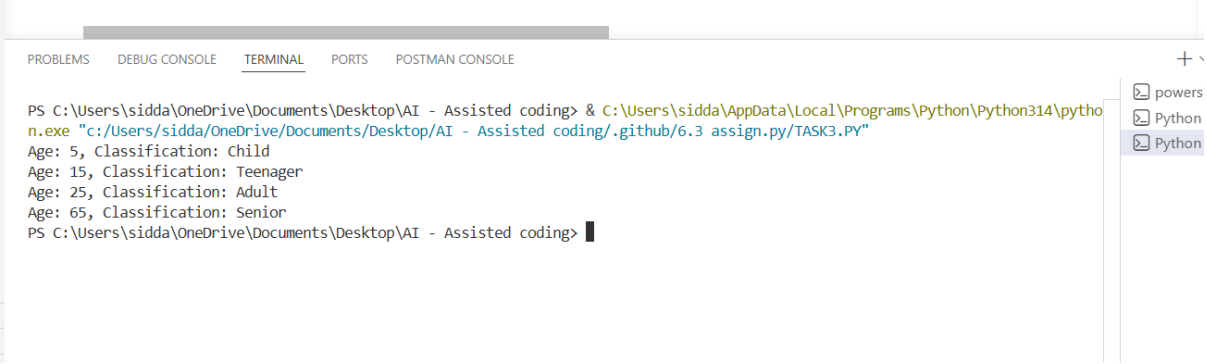
Write a Python function that classifies a person's age into child, teenager, adult, or senior using if-elif-else statements.

CODE:

```
def classify_age(age):
    if age < 0:
```

```
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teenager"
    elif age <= 64:
        return "Adult"
    else:
        return "Senior"
# Example usage
ages = [5, 15, 30, 70, -1]
for age in ages:
    classification = classify_age(age)
    print(f"Age: {age}, Classification: {classification}")
```

OUTPUT:



JUSTIFICATION:

This code defines a function classify_age that takes a person's age as input and classifies it into one of four categories: child, teenager, adult, or senior. The function uses if-elif-else statements to determine the appropriate classification based on the age. The example usage demonstrates how to call the function with different ages and print the classifications.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- **Use AI assistance to generate a sum_to_n() function using a for loop.**
- **Analyze the generated code.**
- **Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.**

Expected Output #4

- **Python function to compute the sum of first n numbers.**

• Correct output for sample inputs.
• Explanation and comparison of different approaches.

PROMPT:
Write a Python function sum_to_n(n) that calculates the sum of the first n natural numbers using a for loop. Include sample input and output.

CODE:
```
def sum_to_n(n):
    total_sum = 0
    for i in range(1, n + 1):
        total_sum += i
    return total_sum
# Example usage
n = 10
result = sum_to_n(n)
print(f"The sum of the first {n} natural numbers is: {result}")
```

OUTPUT:
```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/6.3 assign.py/TASK4.PY"
The sum of the first 10 natural numbers is: 55
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

JUSTIFICATION:
This code defines a function sum_to_n that takes a positive integer n as input and calculates

Task Description #5: Classes (Bank Account Class)
Scenario
You are designing a basic banking application.
Task
• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),
and check_balance().
• Analyze the AI-generated class structure and logic.
• Add meaningful comments and explain the working of the code.
Expected Output #5
• Complete Python Bank Account class.
• Demonstration of deposit and withdrawal operations with updated balance.
• Well-commented code with a clear explanation
PROMPT:

Create a Python class BankAccount with attributes account_holder and balance. Add methods deposit(amount), withdraw(amount), and check_balance(). Include example usage and print outputs. Add comments explaining the code.

CODE:

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        elif amount <= 0:
            print("Withdrawal amount must be positive.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

    def check_balance(self):
        print(f"Current balance for {self.account_holder}: {self.balance}")
# Example usage
account = BankAccount("John Doe", 1000)
account.check_balance()  # Check initial balance
account.deposit(500)     # Deposit money
```

OUTPUT:

```
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/6.3 assign.py/TASK5.PY"
Current balance for John Doe: 1000
Deposited 500. New balance: 1500
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```